OR Spectrum (2017) 39:473–503
DOI 10.1007/s00291-016-0461-z

REGULAR ARTICLE

Gantry crane and shuttle car scheduling in modern rail–rail transshipment yards

Stefan Fedtke[1] · **Nils Boysen**[1]

Received: 14 August 2015 / Accepted: 23 August 2016 / Published online: 1 September 2016

Abstract Modern rail–rail transshipment yards serve as hub nodes in railway networks and enable a rapid consolidation of containers among freight trains. The container transshipment is executed by parallel gantry cranes supported by a sorting system, where shuttle cars preposition containers to relieve the cranes from excessive movement along the spread of the yard. An important decision problem in this context is the scheduling of the gantry cranes, which is considerably complicated by the need to synchronize cranes and shuttle cars whenever container moves are executed via the sorting system. We formalize the resulting interdependent crane and shuttle car scheduling problem, introduce suited solution algorithms, and apply them in order to explore important managerial aspects. Our main findings in this regard are that the position of the sorter either in the middle of the tracks or at the outside has negligible impact on yard performance, while it is a challenging task to match the number of cranes with an appropriate fleet size of shuttles.

Keywords Container logistics · Railway terminals · Crane scheduling · Multiple traveling salesman problem

✉ Nils Boysen
nils.boysen@uni-jena.de
http://www.om.uni-jena.de/

Stefan Fedtke
stefan.fedtke@uni-jena.de
http://www.om.uni-jena.de/

[1] Friedrich-Schiller-Universität Jena, Lehrstuhl für Operations Management, Carl-Zeiß-Straße 3, 07743 Jena, Germany

1 Introduction

Ever since 1955 when Delta Air Lines pioneered the hub-and-spoke paradigm (see Delta 2013), consolidating shipments in a few centralized nodes of a transportation network has emerged as a major logistics strategy for nearly any transportation device. Hub airports bundle inland passengers for moving them overseas in large aircrafts, in central ports containers are consolidated between feeder ships and large intercontinental vessels, and in cross-docking terminals smaller shipments are merged to full truckloads. Only the rail-based freight transport struggles with successfully implementing hub-and-spoke networks. A symptom of this failure is the dramatic decline of the percentage of freight traffic moved by train over the past decades, which in Europe, for instance, fell from 20 % in 1970 to only 10 % in 2005 (EU 2007). Naturally, there are more factors, e.g., the fragmentation of Europe's railway network, that contributed to this decline. A detailed description of this development is, for instance, provided in Vahrenkamp (2010).

One important reason for rail traffic still being predominantly executed as point-to-point traffic (Trip and Bontekoning 2002) is that existing railway terminals are often ill-suited for a rapid transshipment of containers between freight trains. Traditional shunting yards require a complete reassembly of trains in a tedious process via classification tracks and a shunting hill (e.g., see Boysen et al. 2012), with the result that 10–50 % of the trains' total transportation time is consumed by the reassembly process (Bontekoning and Priemus 2004). In intermodal transshipment yards, the train assembly remains unaltered and only the containers are moved by huge gantry cranes spanning truck lanes and railway tracks (e.g., see Boysen et al. 2013). However, if containers are exchanged between trucks and trains, each truck can park directly next to its respective railcar, whereas a container exchange between different trains may require oblong longitudinal container movements along the yard. This causes plenty of interference among gantry cranes which share a special rail track for their longitudinal movement, so that evasion moves are required and train processing is slowed down. Therefore, a new terminal generation has been developed in order to enable rapid transshipment processes among freight trains and, thus, a rail-based hub-and-spoke transport.

1.1 Operations in modern rail–rail transshipment yards

In modern rail–rail transhipment yards—the recent survey paper of Boysen et al. (2013) also denotes them as third generation rail terminals—huge gantry cranes span rail tracks, truck lanes, an intermediate storage area, and a sorting system as is schematically depicted in Fig. 1a. The former elements are also part of traditional transshipment yards applied for intermodal transport in order to transship containers between trucks (driving and parking on the truck lanes) and freight trains (served on the rail tracks). Note that the intermediate storage area is applied, whenever containers need to be double-handled because their delivering and receiving vehicles are not simultaneously present in the terminal. This traditional terminal structure is extended by an additional sorting system in modern rail–rail transshipment yards.

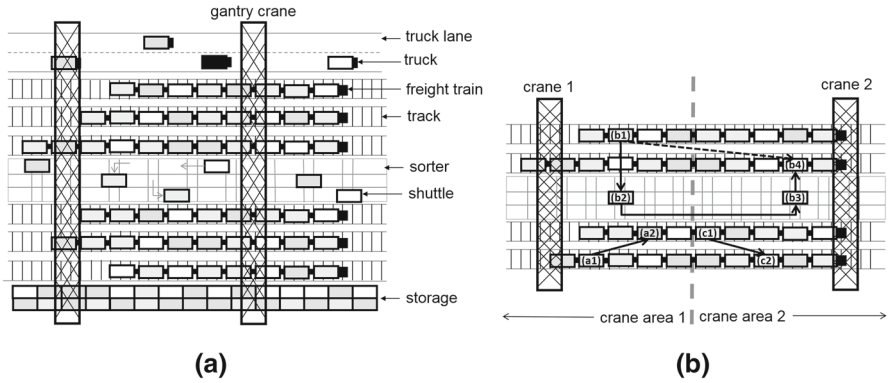

Fig. 1 Layout and organization of modern rail–rail transshipment yards

A sorter consists of special rail tracks for multiple shuttle cars. Shuttle cars are special self-propelled railcars with a capacity for a single container and it is their task to relieve the gantry cranes from long-distant container moves along the spread of the yard. Transferring a container from its inbound railcar to another railcar of another train may require a long movement of the crane along the spread of the yard. The load plan (see Boysen et al. 2013), which defines the positions of containers on the inbound and outbound trains, has to consider plenty aspects, such as weight restrictions, safety margins, and aerodynamic aspects, so that it cannot guarantee that delivering and receiving wagons are always located close by. Long-distant container moves, however, prolong the transshipment process and cause interference among cranes, because evasion moves of other cranes may become necessary on the track shared by the cranes. Thus, long-distant moves are taken over by shuttle cars, so that the cranes can concentrate on a quick container exchange between close-by shuttles and wagons.

The relief by the shuttles is to be traded off against the double handling of containers. The decision whether it is more efficient to directly transfer a container from train to train with a single crane move, which we call a *direct move*, or to intermediately apply a shuttle car in a *split move* is typically decided by partitioning the yard into disjunct crane areas (see Boysen et al. 2010). Each crane area is exclusively operated by a single crane and all container moves having start and target wagon within the same crane area are executed as a direct move, such as container moves (a) and (c) in Fig. 1b. If start and target container are in different areas (move (b) in Fig. 1b) crane interference is avoided by intermediately applying a shuttle for the longitudinal movement along the yard, which results in a split move and, thus, two tedious pick-up and set-down operations of two different cranes. Obviously, split moves considerably complicate the sequencing of the crane moves, because they necessitate an interdependent scheduling of all cranes and shuttles.

1.2 Decision problems and literature review

A recent survey paper summarizing all strategic and operational planning tasks arising in rail terminals is provided by Boysen et al. (2013). For the modern rail–rail transship-

ment yards treated in the paper on hand they identify the following basic operational decision problems:

1. Train bundling: the total set of freight trains to be served during a shift or day needs to be partitioned into successive bundles of trains each simultaneously served on the parallel tracks of the yard. This special partition problem has to consider different arrival times and due dates of trains and should bundle especially those trains together that exchange plenty containers (see Boysen et al. 2011, 2012).
2. Train parking: Each train of a bundle is to be assigned a parking position, i.e., a rail track and a longitudinal stop position along its track (see Alicke and Arnold 1998; Kellner et al. 2012). The parking positions of each bundle should be chosen such that the resulting distances of container moves are minimized.
3. Load planning: the container positions on the train wagons are to be determined, which constitutes a complex decision task due to the multitude of different weight, length, and safety restrictions to be considered (e.g., see Bruns and Knust 2012; Bruns et al. 2014; Ambrosino and Siri 2015). Interdependent load plans for a bundle of trains simultaneously served, such that the distances of container transfers among them are minimized, are derived by Bostel and Dejax (1998).
4. Crane assignment: this problem requires an assignment of container moves to either a single crane (direct move) or to a crane pair (split move). Typically, this decision is made with the help of fixed and disjunct crane areas (see above). Fixed areas have the advantage that crane interference cannot occur, because each crane serves its dedicated area. However, it is also possible to exploit additional flexibility and to allow the cranes to freely move along the yard. In this case, the non-crossing of cranes needs to be ensured by monitoring the crane positions over the complete planning horizon. We presuppose fixed crane areas, which are the typical choice in real-world yards (see Boysen and Fliedner 2010; Boysen et al. 2010, 2013).
5. Crane sequencing: once each crane has received a fixed set of container moves in its respective area, the sequence of these moves is to be determined for each single crane. Unfortunately, these sequences depend on each other because of the split moves. Each split move is to be set into the sorter and moved by its shuttle vehicle first, before it can, finally, be stacked onto its target railcar by another crane.
6. Sorter scheduling: finally, the assignment of shuttle cars to split moves, their exact paths through the sorter and the processing intervals to timely serve the cranes are to be determined.

This paper investigates the latter two problems in a holistic problem setting, so that gantry crane sequences and shuttle schedules are determined. Only very few papers have already treated the scheduling of gantry cranes and shuttles in railway yards. Traditional intermodal yards are, for instance, treated by Froyland et al. (2008), Montemanni et al. (2009), Souffriau et al. (2009), and Kellner and Boysen (2015). All of them derive specific crane scheduling procedures to execute the container transfer between trains, trucks, and intermediate storage. Naturally, these papers do not consider split moves and a sorting system, so that the main complicating aspect of rail–rail transshipment yards is not covered here. The paper of Martinez et al. (2004) treats the transshipment of containers between trains at the border between France and Spain, where different track gauges require a complete transfer of containers. They,

however, do not consider load plans and assume that any container can be put on any wagon. Thus, containers can always be transferred between neighboring railcars, which leads to very simple crane routes easily solvable by rule-based approaches. The gantry crane scheduling in a rail–rail terminal currently built in Germany (called Megahub) is treated by Alicke (2002). This paper does not apply disjunct crane areas, so that an ongoing supervision of the non-crossing of cranes is required. This results in a very complex problem setting integrating decisions 4, 5, and 6, which is formulated as a constraint satisfaction problem with simple heuristic rules for fixing variables. We focus the problems 5 and 6, the interdependent sequencing of cranes and shuttles, for given crane areas. Compared to Alicke (2002) this leads to a more handy problem, while still considering all specifics arising from the sorting system and split moves.

The crane scheduling problem formulated in this paper resembles a special asymmetric multiple traveling salesman problem (mATSP), because a sequence of container moves has to be determined for each crane and each shuttle car. An overview of formulations, applications and solution procedures of the mTSP is given by Bektas (2006). In the mTSP, however, the multiple salesman can facultatively distribute the cities among them. This is not possible in our problem, where the fixed crane areas predetermine the set of jobs to be executed by each crane. Additionally, precedence constraints may occur within a crane area, if a container blocks the target wagon of another container and therefore has to be moved first. Thus, the sequential ordering problem (see, e.g., Gambardella and Dorigo 2000), which extends the traveling salesman problem by precedence constraints, is also related to our problem. However, the complex synchronization constraints among the individual sequences required in our setting are not considered in both streams of the literature (i.e., mTSP and sequential ordering).

Synchronization is rather an emerging topic in the field of vehicle routing problems (VRP) (e.g., Toth and Vigo 2002). Drexl (2012) surveys VRPs with alternative synchronization constraints and denotes our type of synchronization as *exact operation synchronization with precedence constraints*. Similar constraints also occur in related fields such as combined vehicle routing and scheduling problem (VRSP) (e.g., Bredström and Rönnqvist 2008) or the workforce scheduling and routing problem (WSRP) surveyed by Castillo-Salazar et al. (2016). However, all these existing approaches are not directly applicable to our problem.

There also exist other areas of application where multiple cranes need to be coordinated. In large seaports, often multiple quay cranes are applied in parallel to (un-)load container vessels. These cranes also share a rail track for their longitudinal movement along the quay, so that non-crossing constraints are relevant. Survey papers on the resulting quay crane scheduling problem are provided in Bierwirth and Meisel (2010, 2015). Similar crane scheduling problems arise in the container yards of seaports, which intermediately store containers to buffer seaside and hinterland traffic. Many yards apply two cranes to jointly operate a block of containers, so that again non-crossing constraints are relevant. These and other applications of crane scheduling with spatial interference are surveyed in Boysen et al. (2014). The main difference to our problem setting, however, is that these related applications do not face split moves.

It can be concluded that no suited solutions procedures for our gantry crane and shuttle car scheduling problem exist yet.

1.3 Research tasks and paper structure

This paper is dedicated to the interdependent gantry crane and shuttle car scheduling in modern rail–rail transshipment yards, i.e., problems 5 and 6 in Sect. 1.2. For a given yard layout with disjunct crane areas and a given set of container moves with defined start and target wagons we determine the interdependent schedules for all gantry cranes and shuttle cars. Specifically, the split moves into the sorter by one crane, the movement of the sorter vehicle, and the container transfer out of the sorter are to be synchronized. Our first research task is to formalize the resulting crane scheduling problem, derive suited solution procedures, and test their performance in a comprehensive computational study.

Our second aim is to investigate important managerial aspects related to the yard layout. Specifically, we apply our solution procedures to different problem instances and explore the interdependency between the crane number, the fleet size of shuttles and the overall yard performance. Furthermore, we investigate the best position of the sorter, i.e., either in the middle between the rail tracks or at the outside.

A modern rail–rail terminal is, for instance, currently being built in Hannover-Lehrte (Germany). The so-called Megahub applies an additional sorting system and is intended to transship containers between freight trains servicing the North Sea harbors and continental Europe. The gantry crane and shuttle car scheduling procedures developed in this paper are designed to be applicable in terminals like the Megahub.

The remainder of the paper is structured as follows. Section 2 defines our crane scheduling problem in detail and settles its computational complexity. Section 3 introduces a heuristic decomposition procedure, whose solution performance is tested in Sect. 4. Managerial aspects are addressed in Sect. 5 and, finally, Sect. 6 concludes the paper.

2 The gantry crane and shuttle car scheduling problem

2.1 Problem definition

Our interdependent gantry crane and shuttle car scheduling problem aims to minimize the total processing time for transshipping containers among a train bundle parked in a terminal. For a given set of container moves, we seek the sequences of moves processed by each crane and the movements of shuttle cars, such that the completion time of the last crane, i.e., the makespan of train processing, is minimized. This decision task is complicated by the need to synchronize the processing of split moves, which necessitates that all crane sequences and shuttle movements are determined in a coordinated manner. In relation to the decision hierarchy presented in Sect. 1.2, our problem formulation is based on the following assumptions:

- We presuppose that the set of container transfers to be executed for the current bundle of trains is already defined and each container move has a given start and target railcar. When executing our crane sequencing problem, we, thus, assume that the bundle of trains parked in the yard is already determined (problem 1 of

Sect. 1.2), each train is assigned an exact parking position (problem 2), and the load plan of inbound and outbound trains is already defined (problem 3).

- The rail–rail terminals we consider are typically operated under a fixed border policy, so that we assume the yard being partitioned into disjunct crane areas each operated by a dedicated crane (problem 4 of Sect. 1.2). For a given bundle of parked trains and given load plans, fixed area borders define the division of labor among cranes and specify each crane's set of container moves to be executed. All container moves having start and target wagon in the same crane area are executed as a direct move by the one crane operating the respective area. Start and target wagons spread out over different areas require a split move, each consisting of three different parts. The crane of the start area executes the *in-move* from the start railcar onto a shuttle car, the shuttle movement within the sorter is denoted as the *shuttle-move*, and the crane of the target area, finally, executes the *out-move* from the shuttle car to the target wagon.
- With regard to the sorting system, we consider a fixed shuttle fleet containing a given number of identical shuttle cars. We explicitly model which shuttle-move is executed by which shuttle and consider the processing times of loaded and unloaded shuttle moves. However, we introduce two simplifying assumptions. First, we assume that in- and out-moves of cranes are always executed without longitudinal crane movement, so that the containers are always moved to and from the sorter position under the respective railcar (see Fig. 1b). This way, we have fixed distances to be covered by all loaded and unloaded moves. Due to their huge investment cost gantry cranes are often considered as the bottleneck resource in a terminal, so that the policy of reducing the cranes' workload as much as possible seems a reasonable choice. Furthermore, we assume that no shuttle interference occurs within the sorter, so that given the shuttles' velocity all processing times of sorter moves can be calculated. This assumption seems a fair approximation of reality for the sorter once planned for the aforementioned Megahub (see Fig. 2). This sorter consists of a rectangular system of (outer) sorter tracks dedicated to the mono-directional movement of shuttles. Shuttles waiting to be (un-)loaded by some crane are only allowed in the waiting bays on the inside of the sorter. Note that the shuttles are designed to have rotatable wheels, which they turn by 90° when moving into a bay or onto another sorter track (see Rotter 2004). Such a mono-directional travel of shuttles rules out most potential interference. Only when trying to leave a waiting bay, a shuttle may be delayed by another shuttle just passing by. However, the number of shuttles applied in a sorter does typically not exceed a dozen, so that the probability for these delays of just a few seconds is low.

  Both aforementioned simplifying assumptions allow us to calculate processing times of shuttle-moves and the loaded movement between two successive jobs upfront in a preprocessing step. Integrating the detailed shuttle movement to consider (schedule-dependent) delays require much more complex models, which are left to future research.

Given these assumptions, we have a set $J = \{1, \ldots, n\}$ of container moves to be executed by m parallel gantry cranes and a sorting system consisting of o identical shut-

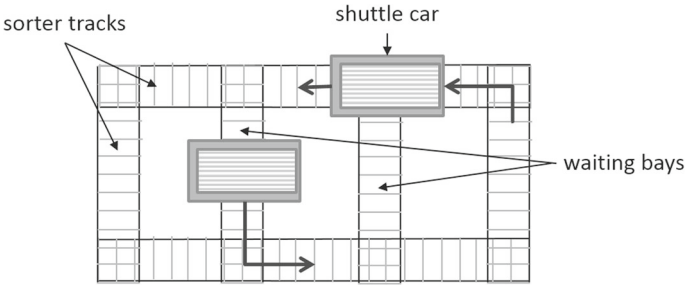

Fig. 2 Schematic layout of a sorter

tle vehicles, which are represented by the sets $C = \{1, \ldots, m\}$ and $V = \{1, \ldots, o\}$, respectively. Note that the job set contains all direct moves and all three partial moves, i.e., an in-move, shuttle-move, and out-move for each split move. We apply in(j) and out(j) to refer to the in- and out-move of a shuttle-move j, respectively. Thus, job set J consists of moves to be executed by some crane, which we denote as crane job set J^C, and the shuttle moves J^V. Thanks to the fixed crane areas, we can partition job set J^C into sets $J_1, J_2, \ldots, J_m$ containing the crane moves fixedly assigned to each crane $c = 1, \ldots, m$.

Each move $j \in J$ is assigned a processing time p_j and the empty movement between two jobs i and j is represented by "setup time" τ_{ij}. For a crane move $j \in J^C$ processing time p_j covers the time span from the crane's pick-up operation at the start position of the move (either its start railcar or a shuttle vehicle in the sorter), to move to the target position (either its target railcar or the sorter), and to set down the box. Then, a crane has to move empty from the target position of current move i to the start position of successor job j, which takes τ_{ij} time units. For a shuttle move $j \in J^V$ processing time p_j equals the waiting time during loading a container, the movement time towards the target area, and the unloading time. The empty travel between the target position of predecessor job i and the start position of successor job j is represented by τ_{ij}. Whenever a container is loaded by some crane onto a shuttle car or vice versa both transportation devices have to be simultaneously present to execute the loading or unloading operation. We presuppose that this time span varies between pick up (δ^{out}) and set down (δ^{in}).

Finally, we have to address additional restrictions. Precedence constraints may occur between two crane jobs executed by the same crane. Whenever, a container j has a target railcar, which is still occupied by another container i, then i has to be removed from the wagon first and is, thus, contained in predecessor set Γ_j. Furthermore, we consider initial and final crane positions before each crane has started and after it has finished job processing, respectively. We introduce dummy job $j = 0$ with zero processing time, so that the movement from its initial position to the start position of the first job j (from the target position of the last job j to its final position) can be assigned to τ_{0j} (τ_{j0}). For the shuttles we do not consider initial and target positions and assume that there is always enough time for moving towards their first site of operation during the respective in-move of the crane. However, adding initial and

target positions as part of the input data for each shuttle is truly straightforward, so that for a matter of conciseness we do without a detailed description.

Given these explanations, our gantry crane and shuttle car scheduling problem in modern rail–rail transshipment yards (denoted as CSCSP) can be formally defined as follows. A schedule Ω for the overall system contains the schedule information for all crane jobs $j \in J^C$ and for all shuttle jobs $j \in J^V$. The former is specified by a tuple $(j, e_j) \in \Omega$, which defines the completion time e_j to each crane job $j \in J^C$. The execution of a shuttle job $j \in J^V$ is specified by a quadruple $(j, s_j, e_j, v) \in \Omega$ where s_j and e_j define the points in time where the first crane has just started setting down box $j \in J^V$ onto shuttle vehicle $v \in V$, i.e., shuttle job j's start time, and the target crane has just finished picking up the container in its target area, i.e., shuttle job j's completion time, respectively. We say a schedule Ω is feasible, if

- for each $i \in J^C$ and each $j \in J^V$ there is exactly one $(i, e_i) \in \Omega$ and one $(j, s_j, e_j, v) \in \Omega$, respectively, that is all jobs are processed exactly once;
- for each $(j, e_j) \in \Omega$ with $j \in J^C$ we have $e_j \geq \tau_{0j} + p_j$, that is a crane job cannot finish before the respective crane has moved from its initial position (encoded by dummy job 0) to job j plus its processing time p_j;
- for each pair of crane jobs $(i, e_i) \in \Omega$ and $(j, e_j) \in \Omega$ with $i \neq j$ and $i \in \Gamma_j$ that are executed by the same crane c we have $e_j \geq e_i + \tau_{ij} + p_j$, that is precedence constraints of jobs executed by the same crane have to be considered;
- for each pair of crane jobs $(i, e_i) \in \Omega$ and $(j, e_j) \in \Omega$ with $i \neq j$ and neither $i \in \Gamma_j$ nor $j \in \Gamma_i$ that are executed by the same crane c we have either $e_j \geq e_i + \tau_{ij} + p_j$ or $e_i \geq e_j + \tau_{ji} + p_i$, that is jobs of the same crane without precedence constraints must not have processing intervals that overlap;
- for each $(j, s_j, e_j, v) \in \Omega$ we have $e_j \geq s_j + p_j$, that is each shuttle job takes at least its processing time plus optional waiting time;
- for each pair of shuttle jobs $(i, s_i, e_i, v) \in \Omega$ and $(j, s_j, e_j, w) \in \Omega$ with $i \neq j$ we have either $v \neq w$ or $s_j \geq e_i + \tau_{ij}$ or $s_i \geq e_j + \tau_{ji}$, that is jobs are either executed by different shuttles or their processing intervals do not overlap, and
- for each shuttle job $(j, s_j, e_j, v) \in \Omega$ we have $e_{\text{in}(j)} - \delta^{\text{in}} = s_j$ and $e_{\text{out}(j)} = e_j - \delta^{\text{out}} + p_{\text{out}(j)}$, that is in-move in$(j)$, shuttle move j, and out-move out(j) (altogether constituting a specific split move) have to be synchronized.

Among all feasible schedules CSCSP seeks one Ω with minimum makespan, i.e., a schedule that minimizes $Z(\Omega) = \max_{j \in J}\{e_j + \tau_{j0}\}$.

Example: Consider the yard layout and the given container transfers depicted in Fig. 1b. For calculating the processing times p_j and setup times τ_{ij} we presuppose equally sized containers, so that the yard can be subdivided into a grid (see Fig. 3). We suppose that cranes and shuttles move by one field per time unit and pick-up and set-down times δ^{in} and δ^{out} are negligible. Typically, cranes and crane trolleys, which move along their crane's beam in lateral direction, are propelled with different engines, so that the maximum metric is to be applied for calculating the movement times of cranes. For instance, the execution of job (a) requires a movement of two tiles in longitudinal direction and one tile in lateral direction, so that processing time amounts to $p_a = \max\{2; 1\} = 2$. Figure 3 depicts the optimal solution of CSCSP with $Z = 14$. Crane 1 starts from its initial position in the top left tile, executes in-move (b), then direct move

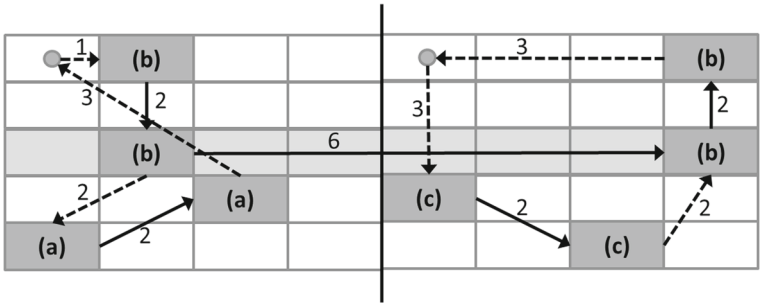

Fig. 3 Optimal solution for the example

(a), and returns to its end position, which takes $p_0 + \tau_{0,b} + p_b + \tau_{ba} + p_a + \tau_{a0} = 0 + 1 + 2 + 2 + 2 + 3 = 10$ time units. Crane 2 starts in the top left position of its crane area, executes direct move (c) and then (b). However, it cannot instantaneously start executing the out-move of split move (b) when reaching the respective tile over the sorter after 7 time units. It has to wait for the arrival of the shuttle car after 9 time units. Only afterwards, the crane can proceed with (b) and return to its end position after 14 time units.

After having defined our CSCSP, a natural next question is the one for its computational complexity.

Theorem 1 *CSCSP is strongly NP-hard.*

Proof See Appendix A. □

2.2 Mixed-integer model

This section provides a mixed-integer model for CSCSP. Given the notation summarized in Table 1, CSCSP can be formulated as mixed-integer program CSCSP-MIP as follows:

$$\text{CSCSP-MIP: } Z(S, E, W, X, Y, z) = z \quad \rightarrow \quad \min \tag{1}$$

subject to

$$z \geq e_j + \tau_{j,n+c} \qquad \forall c \in C;\ j \in J_c \tag{2}$$

$$\sum_{i \in J_c : i \neq j} x_{ij} = 1 \qquad \forall c \in C;\ j \in J_c \tag{3}$$

$$\sum_{j \in J_c : i \neq j} x_{ij} = 1 \qquad \forall c \in C;\ i \in J_c \tag{4}$$

$$e_j \geq e_i + \tau_{ij} + p_j \qquad \forall j \in J^C \setminus D^C;\ i \in \Gamma_j \tag{5}$$

$$e_j \geq e_i + \tau_{ij} + p_j - M \cdot (1 - x_{ij}) \qquad \forall c \in C;\ i \in J_c;\ j \in J_c \setminus \{n + c\} \tag{6}$$

$$e_j \geq 0 \qquad \forall j \in D^C \tag{7}$$

Table 1 Notation for CSCSP

J	Set of container moves ($J = \{1, 2, \ldots, n\}$)
C	Set of cranes ($C = \{1, 2, \ldots, m\}$)
V	Set of shuttles/sorter vehicles ($V = \{1, 2, \ldots, o\}$)
D^C	Set of crane dummy moves ($D^C = \{n+1, n+2, \ldots, n+m\}$)
D^V	Set of shuttle dummy moves ($D^V = \{n+m+1, n+m+2, \ldots, n+m+o\}$)
J^C	Set of crane moves (with $D^C \subset J^C$)
J^V	Set of shuttle moves (with $D^V \subseteq J^V$)
J_c	Set of container moves dedicated to crane c (including dummy move $n+c$)
$\text{in}(j)$	In-move dedicated to shuttle move $j \in J^V \setminus D^V$
$\text{out}(j)$	Out-move dedicated to shuttle move $j \in J^V \setminus D^V$
Γ_j	Set of predecessors of crane move $j \in J^C \setminus D^C$
p_j	Processing time of move $j \in J^C \cup J^V$
τ_{ij}	$\begin{cases} \text{empty crane moving time} & \exists c \in C : i, j \in J_c \wedge i \neq j \\ \text{empty sorter moving time} & i, j \in J^V \wedge i \neq j \\ 0 & \text{else} \end{cases}$
δ^{in}	Shuttle loading time
δ^{out}	Shuttle unloading time
M	Big integer, e.g., $\lceil \sum_{i,j \in J^C \cup J^V} \tau_{ij} + \sum_{j \in J^C \cup J^V} p_j \rceil$
w_{jv}	Binary variable: 1, if move j is processed by vehicle v; 0, otherwise ($j \in J^V, v \in V$)
x_{ij}	Binary variable: 1, if crane move i is executed before j; 0, otherwise ($i, j \in J^C$)
y_{ij}	Binary variable: 1, if sorter move i is executed before j; 0, otherwise ($i, j \in J^V$)
s_j	Continuous variable: start time of shuttle move j ($j \in J^V$)
e_j	Continuous variable: ending time of move j ($j \in J^C \cup J^V$)
z	Continuous variable: makespan

$$\sum_{i \in J^V} y_{ij} = 1 \qquad \forall j \in J^V \tag{8}$$

$$\sum_{j \in J^V} y_{ij} = 1 \qquad \forall i \in J^V \tag{9}$$

$$s_j \geq e_i + \tau_{ij} - M \cdot (1 - y_{ij}) \qquad \forall i \in J^V; j \in J^V \setminus D^V \tag{10}$$

$$s_j \geq 0 \qquad \forall j \in D^V \tag{11}$$

$$e_j \geq 0 \qquad \forall j \in D^V \tag{12}$$

$$1 - y_{ij} \geq w_{iv} - w_{jv} \qquad \forall v \in V; i, j \in J^V \tag{13}$$

$$1 - y_{ij} \geq w_{jv} - w_{iv} \qquad \forall v \in V; i, j \in J^V \tag{14}$$

$$\sum_{v \in V} w_{jv} = 1 \qquad \forall j \in J^V \tag{15}$$

$$w_{j,j-n-m} = 1 \qquad \forall j \in D^V \tag{16}$$

$$e_j \geq s_j + p_j \qquad \forall j \in J^V \tag{17}$$

$$e_{\text{in}(j)} - \delta^{\text{in}} = s_j \qquad \forall j \in J^V \setminus D^V \tag{18}$$

$$e_{\text{out}(j)} = e_j - \delta^{\text{out}} + p_{\text{out}(j)} \qquad \forall j \in J^V \setminus D^V \tag{19}$$

$$x_{ij} \in \{0, 1\} \qquad \forall i, j \in J^C \tag{20}$$

$$y_{ij} \in \{0, 1\} \qquad \forall i, j \in J^V \tag{21}$$

$$w_{jv} \in \{0, 1\} \qquad \forall j \in J^V; v \in V \tag{22}$$

Objective function (1) seeks to minimize the completion time of all cranes, i.e., the makespan, which is defined in (2). Equations (3)–(7) define the job sequences of cranes and (8)–(9) the sequences for shuttle cars. Thereby, equations (3) and (4) (8) and (9) ensure, that each crane (shuttle) job is processed exactly once. Precedence relations for crane and shuttle moves are considered in (5), (6) and (10). Note that constraints (6) and (10), moreover, ensure that two jobs cannot be (direct) predecessor and successor simultaneously. Start and completion times of dummy moves (and hence for all other moves) are declared non-negative within equations (7), (11) and (12). Moreover, inequalities (13) and (14) guarantee that consecutive sorter moves are scheduled on the same shuttle and (15) and (16) assure, that each sorter move is assigned to exactly one vehicle. Constraints (17) ensure, that each shuttle job takes at least its processing time plus optional waiting time. Inequalities (18) and (19) connect crane and sorter sequences by scheduling sorter moves and their designated in- and out-moves. Finally, constraints (20)–(22) define the domain of all binary variables.

3 A decomposition procedure for solving CSCS

Synchronizing all cranes and shuttles turns out to be a challenging task, so that decomposing the problem into more handy subproblems seems a worthy endeavor. Our decomposition procedure makes use of the fact that once all shuttle-moves receive a fixed processing interval determining the remaining part of the sorter schedule, i.e., assigning a shuttle to each move, and sequencing the remaining direct moves of each crane can be executed independent of each other. This basic idea is applied multiple times during our decomposition procedure, which is subdivided into the following three basic stages.

1. Construct an initial feasible solution by applying a construction heuristic (CH) based on priority rules.
2. Fix the processing intervals of all split moves $j \in J^V$ from Stage 1's solution and improve the resulting independent ATSPs for each crane by a dynamic programming (DP) based procedure.
3. Repeatedly solve the mode feasibility problem (MFP) with an off-the-shelf solver to fix the processing intervals of split moves and optimize the crane schedules with DP.

All stages are described in greater detail in the following sections.

3.1 Stage 1: construction heuristic

In stage one, we introduce a straightforward multi-start heuristic procedure to construct a first feasible solution for CSCSP. Our construction heuristic (CH) applies basic priority rules such as *nearest neighbor* and *FCFS* when successively adding moves to cranes and shuttle ranked according to their (current) smallest completion time (see Algorithm 1).

Algorithm 1: Construction heuristic (CH)

input: processing times p_j, moving times τ_{ij}, and precedence relations of moves
start with empty crane and shuttle sequences
assign the respective dummy move to each crane
repeat
  determine feasible moves for each crane
  determine the crane with smallest current completion time having feasible moves
  determine next available shuttle (according to smallest completion time)
  determine next available move in crane sequence randomly (based on completion time)
  assign move to crane sequence
  if *in-move scheduled* **then**
    schedule designated sorter-move
  end
  update crane and shuttle sequences, starting and completion times and makespan
until *all moves are assigned*;
return: feasible move sequences for cranes and shuttles, makespan

Initially, we set start times s_j and completion times e_j to zero for each job $j \in J \cup \{0\}$ and introduce the following sets:

- $SC_c = \{0\}$, already scheduled crane moves of crane c,
- $SV_v = \emptyset$, already scheduled shuttle moves of shuttle v and
- $FEAS_c = \emptyset$, currently feasible crane moves of crane c.

During each iteration of CH one or two moves are added to the sets of already scheduled moves. Note that once CH is executed we obtain $SC_c = J_c \cup \{0\}$ and $\bigcup_{v \in V} SV_v = J^V$.

At the beginning of each iteration, we determine all moves that can currently be scheduled and add them to the set of feasible moves for each crane. A direct move j is feasible, if it either has no predecessor (i.e., $\Gamma_j = \emptyset$) or each predecessor of j is already scheduled (i.e., $\exists c \in C, \forall i \in \Gamma_j : i \in SC_c$). An out-move is feasible, if in addition to the condition for the direct moves the dedicated in-move and shuttle-move are already scheduled. In-moves are feasible, if at least one shuttle is currently available (no container loaded). Let $l_c = \text{argmax}_{\text{j} \in \text{SC}_\text{c}} \{\text{e}_\text{j}\}$ be the last move in crane c's sequence. Afterwards, we determine the crane with the smallest completion time

$$c^* = \underset{\substack{c \in C: \\ FEAS_c \neq \emptyset}}{\text{argmin}} \left\{ e_{l_c} \right\}.$$

To determine the next move of crane c^*'s sequence we first detect the next available shuttle v^* having the smallest completion time. If $SV_{v^*} \neq \emptyset$, i.e., shuttle v^* already

has moves assigned, than we denote k_{v^*} as the last move of this vehicle. Subsequently, we calculate the increase of completion time

$$\kappa_j = \begin{cases} \tau_{l_{c^*} j}, & \text{if } j \text{ is a direct move} \\ \tau_{l_{c^*} j}, & \text{if } j \text{ is an in-move and } SV_{v^*} = \emptyset \\ \max\left\{e_{l_{c^*}} + \tau_{l_{c^*} j}, e_{k_{v^*}} + \tau_{k_{v^* i}}\right\} - e_{l_{c^*}}, & \text{if } j = \text{in}(i) \text{ and} SV_{v^*} \neq \emptyset \\ \max\left\{e_{l_{c^*}} + \tau_{l_{c^*} j}, e_i - \delta^{\text{out}}\right\} - e_{l_{c^*}}, & \text{if } j = \text{out}(i) \end{cases}$$

and derive priority values $p_j = \frac{1}{\kappa_j}$ for each move $j \in FEAS_{c^*}$. Among all moves of $FEAS_{c^*}$ we, now, randomly pick move j^* where the choice probability of each move j equals the relation of priority value p_j to the sum over all values. The selected move is, then, added to the sequence of crane c^*. If j^* is an in-move, i.e., $j^* = \text{in}(i)$, we also schedule the designated shuttle-move i for the next available shuttle v^*. Finally, we update the current status: $SC_{c^*} = SC_{c^*} \cup \{j^*\}$, $l_{c^*} = j^*$ and possibly $SV_{v^*} = SV_{v^*} \cup \{i\}$. Furthermore, we determine the new start and completion times for j^* (and i, if necessary). This process is repeated until all moves are scheduled. Due to the random selection process, we can repeat CH ψ times to, finally, return the best solution found as the result of stage 1.

3.2 Stage 2: improving crane schedules with DP

Given the first feasible solution of stage 1, we only fix all in-moves, out-moves and shuttle-moves as is defined by the first given schedule. Once all in- and out-moves to and from the sorter are fixed, the direct moves remain to be scheduled for each crane separately. We obtain m independent ATSPs with some jobs already fixed (i.e., in- and/or out-moves), which can individually be optimized. Moreover, for improving the makespan only the respective bottleneck crane with the highest completion time needs to be optimized.

For this purpose, we describe how to adapt the basic DP scheme for sequencing problems provided by Held and Karp (1962) to our problem and how to apply this scheme in a beam search heuristic. Let the already fixed tuples of the ϕ_c in- and out-moves to be executed by the respective crane c be stored according to increasing completion time e_j within $\Phi_c = \langle (j_1, e_{j_1}), \ldots, (j_{\phi_c}, e_{j_{\phi_c}}), (j_{\phi_c+1}, \infty) \rangle$. Note that the final fixed move within Φ_c denotes a virtual in- or out-move with zero processing time, which is introduced to ease the following descriptions. Given this sequence of already fixed crane moves and completion time t of final move i fixed for the current crane, the earliest start time $\Delta(i, t, j)$ of a not yet fixed direct move j is calculated by

$$\Delta(i, t, j) = \begin{cases} t + \tau_{ij}, & \text{if } t + \tau_{ij} + p_j + \tau_{jk^*} \leq e_{k^*} - p_{k^*} \\ e_{l^*} + \tau_{l^* j}, & \text{else} \end{cases} \tag{23}$$

where k^* and l^* define the next fixed job after t, i.e., $k^* = j_{\min_{q=1,\ldots,\phi_c+1}\left\{q | e_{j_q} > t\right\}}$, and the next fixed job after which job j can be scheduled without interfering with other

fixed jobs, i.e., $l^* = j_{\min_{q=1,\ldots,\phi_c}\{q|(e_{j_q}>t)\wedge(e_{j_q}+\tau_{j_q j}+p_j+\tau_{jj_{q+1}}\leq e_{j_{q+1}}-p_{j_{q+1}})\}}$, respectively. Thereby, we distinguish cases in which j can be scheduled directly after i and cases in which j has to be scheduled after fixed in- or out-moves.

We consider states (J', j) with $J' \subseteq J^*$ and $j \in J'$ where J^* denotes the set of yet unscheduled direct moves of the current crane c, which is defined as $J^* = J_c \setminus (\{\text{in}(j)|j \in J^V\} \cup \{\text{out}(j)|j \in J^V\})$. A state (J', j) is defined by the subset J' of boxes already scheduled and the last move j defining the crane's current position. The minimum time associated with a state is represented by $f(J', j)$. There is a transition from state (J', j) to state (J'', k), if $J'' = J' \cup \{k\}$ and $\Gamma_k \subseteq J'$, i.e., a transition for every possible remaining direct move whose predecessors have already been scheduled. The earliest start time of a job j, which is executed last among subset J' can be calculated by calling function $\Delta(i, f(J'\setminus\{j\}, i), j)$, so that the basic Bellman recursion is

$$f(J', j) = \begin{cases} \Delta(0, 0, j) + p_j \\ \qquad\qquad \text{, if } J' = \{0, j\} \\ \\ \infty \\ \qquad\qquad \text{, if } \exists q \in \{1, \ldots, \phi_c\} : j \in \Gamma_{j_q} \\ \qquad\qquad \text{and } e_{j_q} < \min_{i \in J'\setminus\{0,j\}} \left\{\Delta\left(i, f(J' \setminus \{j\}, i), j\right) + p_j\right\} \\ \\ \min_{i \in J'\setminus\{0,j\}} \left\{\Delta\left(i, f(J' \setminus \{j\}, i), j\right) + p_j\right\} \\ \qquad\qquad \text{, otherwise} \end{cases}$$

$$\forall J' \subseteq J^*, j \in J' \setminus \{0\} \tag{24}$$

with $f(\{0\}, 0) = 0$. Hereby, we differentiate between the initial job 0, predecessors of split moves that cannot be scheduled anymore (if their completion time is larger than their successor's) and all other moves. Recall that job 0 represents a dummy job to integrate the crane's empty movement from its initial position to its first job. We apply (24) in a stage-wise forward recursion until the final stage is reached and all jobs are scheduled. Here, we compare all final states and determine optimal solution value Z by

$$Z = \min_{j \in J^*\setminus\{0\}} \left\{\max\{f(J^*, j) + \tau_{j0};\ e_{j_{\phi_c}} + \tau_{j_{\phi_c}0}\}\right\}. \tag{25}$$

The minimum makespan for crane c is either determined by the completion time of the last direct move ($f(J^*, j)$) plus the return time to the crane's final position (τ_{j0}) or the completion time of the last in- or out-move ($e_{j_{\phi_c}}$) plus the return time ($\tau_{j_{\phi_c}0}$) depending on which of both finishes later. A simple backward recursion can be applied in order to extract the optimum schedule from the optimal state.

The number of states is in $\mathcal{O}(n2^n)$, each state originates up to n transitions, and a transition requires an iteration through all fixed jobs, so that the computational effort of DP is in $\mathcal{O}(n^3 \cdot 2^n)$.

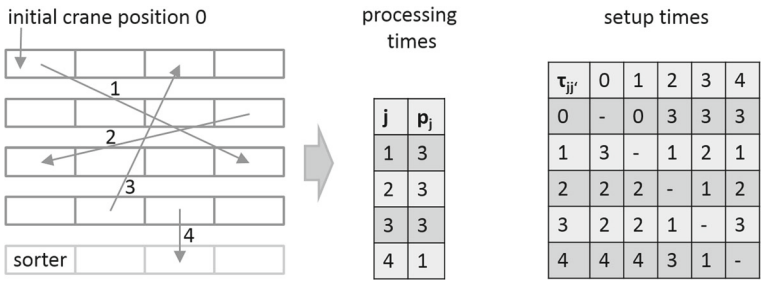

Fig. 4 Crane area with four moves and instance data

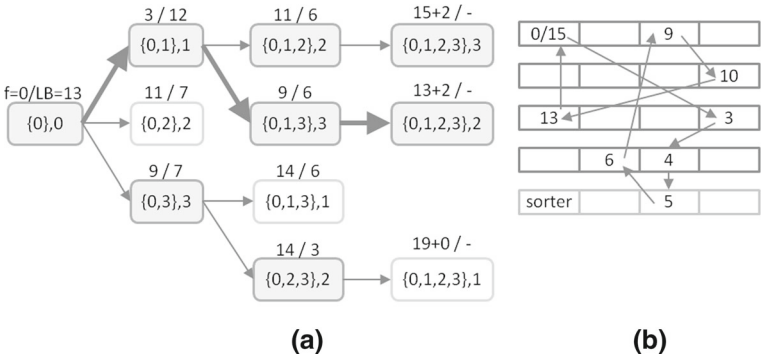

Fig. 5 DP graph and the resulting crane sequence

To accelerate our DP upper and lower bounds can be integrated. A global upper bound UB can be determined upfront by some heuristic procedure. We apply beam search as described in the subsequent paragraph. Whenever a state specific lower bound $LB(J', j)$ plus the shortest path value $f(J', j)$ to current state (J', j) is equal to or exceeds upper bound UB, the respective state can be fathomed as it cannot be part of a solution with a better objective value than the incumbent solution. Our simple lower bound adds up the remaining processing times plus the minimum setup times, so that $LB(J', j) = \sum_{k \in J^* \setminus J'} (p_k + \min_{i \in J^* \setminus \{k\}} \{\tau_{ik}\}) + \min_{k \in J^* \setminus J'} \{\tau_{k0}\}$.

Example: Consider the crane area schematically depicted in Fig. 4. Three direct moves are to be executed and an additional in-move $j = 4$ whose processing interval is already fixedly defined by tuple $(j = 4, e_j = 5)$. For calculating the processing times and setup times we apply the maximum metric and simply count the maximum number of tiles moved in lateral and longitudinal direction. The time for pick-up and set-down operations is assumed to be negligible. These assumptions lead to the instance data depicted on the right hand side of Fig. 4. The resulting DP graph for this example and a given upper bound $UB = 18$ is depicted in Fig. 5a. The bold faced optimal path represents the crane sequence depicted in Fig. 5b. Note, that, for example, $LB = 13$ for the first stage follows from the sum of all processing times $(3 + 3 + 3 = 9)$, the minimum setup times for remaining crane moves $(0 + 1 + 1 = 2)$, and the minimum time to return to the initial position (2).

As the state space of our DP grows exponentially with the number of jobs, we, furthermore, apply our DP scheme in a heuristic beam search approach. Beam search only branches the BW (beam width) most promising states per stage of the DP tree. For a sufficiently small BW, both the space and the runtime required for solving the crane scheduling are polynomially bounded. However, there is no guarantee that the optimal solution is among those considered. To apply beam search, the BW best nodes per stage are selected according to $f(J', j) + LB(J', j)$.

Within stage 2 of our decomposition approach, we apply beam search for improving the schedule of the bottleneck crane. If the completion time of this crane is reduced and another crane becomes the bottleneck, beam search is applied to this crane as well and so on until no further improvement can be realized.

3.3 Stage 3: mode feasibility procedure

Stage 3 applies what we dub the mode feasibility procedure (MFP), which iteratively tries to reschedule the split moves and to optimize the crane schedules. The basic steps of an iteration of stage 3 are:

1. Generate a set of potential processing intervals of each shuttle-move $j \in J^V$ by randomly drawing completion times e_j.
2. Select a processing interval for each shuttle, so that job execution does not overlap by solving MFP with an off-the-shelf solver.
3. Given the fixed in- and out-moves, sequence the remaining direct moves for each crane separately with beam search (see Sect. 3.2).

The only input handed over from the predecessor stage is the makespan of the best solution obtained. All other information is neglected and new shuttle moves are fixed in order to guide the search into another area of the solution space. Obviously, this makespan determines an upper bound and there is no need to generate processing intervals whose completion times exceed the current makespan. Thus, we randomly draw potential modes $\mu \in M_j$ for each split move $j \in J^V$. A mode specifies the point in time a crane sets down the container on some shuttle (therefore the crane must have picked up the box at an appropriate time from its railcar) and another crane picks up the box from the shuttle in the target area (and then finally stacks the box on its dedicated railcar). We define two kinds of incompatibilities:

- *Shuttle incompatibility:* Set $\Psi^{\mathrm{Sh}}_{\mu j}$ contains all incompatible tuples (μ', j') referring to mode μ' of job j', which cannot be executed by the same shuttle as mode μ of job j. That is

$$(\mu' j') \in \Psi^{\mathrm{Sh}}_{\mu j} \Leftrightarrow \left(e_{(\mu', j')} + \tau_{j' j} > s_{(\mu, j)}\right) \wedge \left(e_{(\mu, j)} + \tau_{j j'} > s_{(\mu', j')}\right) \quad (26)$$

  with $s_{(\mu, j)}$ ($e_{(\mu, j)}$) defining the start (completion) time of job j in mode μ. Specifically, these sets contain all job modes whose start positions cannot timely be reached by the same shuttle after executing mode μ of job j.
- *Crane incompatibility:* Set $\Psi^{Cr}_{\mu j}$ contains a set of tuples (μ', j'), which define that mode μ' of job j' cannot be executed together with mode μ of job j, because this

would require some crane to (un-)load shuttles concurrently (first condition) or precedence relations would be violated (second condition). That is

$$(\mu' j') \in \Psi_{\mu j}^{Cr} \Leftrightarrow \begin{array}{r} \exists c \in C, i \in J_c \cap \{in(j), out(j)\}, i' \in J_c \cap \{in(j'), out(j')\} : \\ ((e_{i'}^{(\mu',j')} + \tau_{i'i} > s_i^{(\mu,j)}) \wedge (e_i^{(\mu,j)} + \tau_{ii'} > s_{i'}^{(\mu',j')})) \\ \vee ((i' \in \Gamma_i) \wedge (e_{i'}^{(\mu',j')} > e_i^{(\mu,j)})) \end{array} \tag{27}$$

with $s_i^{(\mu,j)}$ ($e_i^{(\mu,j)}$) defining the start (completion) time of job i, if job j is executed in mode μ.

We have a set V of shuttle cars and define binary variable $\lambda_{\mu j v}$ to receive value 1, whenever shuttle v executes modes μ of job j, and 0 otherwise. The mode feasibility problem (MFP) is defined as follows:

$$\sum_{\mu \in M_j} \sum_{v \in V} \lambda_{\mu j v} = 1 \qquad \forall j \in J^V \tag{28}$$

$$\lambda_{\mu' j' v} + \lambda_{\mu j v} \leq 1 \qquad \forall v \in V;\ \mu \in M_j;\ j \in J^V;\ (\mu', j') \in \Psi_{\mu j}^{Sh} \tag{29}$$

$$\sum_{v \in V} \lambda_{\mu' j' v} + \sum_{v \in V} \lambda_{\mu j v} \leq 1 \qquad \forall \mu \in M_j;\ j \in J^V;\ (\mu', j') \in \Psi_{\mu j}^{Cr} \tag{30}$$

$$\lambda_{\mu j v} \in \{0, 1\} \qquad \forall v \in V;\ \mu \in M_j;\ j \in J^V \tag{31}$$

Constraints (28) ensure that each job is executed in exactly one mode by exactly one shuttle. Constraints (29) avoid that shuttle incompatible modes are executed by the same shuttle and constraints (30) exclude that some crane has to (un-)load shuttles concurrently. Finally, (31) sets the domain of binary variables.

Although the following complexity result is to be attributed to MFP, we hope that this handy feasibility problem is quickly solvable for an off-the-shelf solver if the capacity situation is not too tight. Unfortunately, we still face a complex optimization problem, but at least the multitude of decisions, which, moreover, have to be synchronized in our original CSCSP, is now reduced to a single mode choice per job.

Theorem 2 *MFP is strongly NP-complete even if* $|V| = 1$.

Proof See Appendix B. □

Stage 3 of our decomposition procedure consists of several iterations, in order to find a new (smaller) makespan than given by stage 2. First, we randomly draw a new (current) makespan Z' from the interval $[L, U)$. In the first iteration, L is given by a lower bound for our problem determined with a spanning tree approach and U is the upper bound (makespan) given from stage 2. Next, we randomly draw modes from $[0, Z']$ (Step 1 at the beginning of this section) and apply an off-the-shelf solver to the resulting mode feasibility problem (Step 2). If a feasible solution can be obtained, we perform the DP/BS approach described in stage 2 in order to schedule the remaining direct crane moves (Step 3). For beam search, we first utilize a smaller beam width

SBW to quickly construct a feasible solution and afterward a larger beam width LBW to improve the bottleneck crane. In case a feasible solution with a makespan smaller than U was obtained, we update $U := Z'$ and repeat the process. Otherwise, we again draw a set of modes randomly for a maximum of REP times or increase the number of drawn modes MOD. If, however, no better solution can be obtained, we update $L := Z'$ and start a new iteration. The procedure stops, once $U - L$ drops under a given tolerance TOL.

4 Computational performance

In this section, we investigate the computational performance of our decomposition procedure. First, we elaborate on instance generation and distinguish two cases: *small* instances can be solved to optimality by an off-the-shelf solver and *large* instances represent real-world yards. The parameters for generating our instances are given in Table 2.

Instance generation has been repeated 10 times (small instances) and 3 times (large instances) for each combination of parameters, which results in a total of $10 \cdot 3 \cdot 3 \cdot 5 \cdot 5 = 2250$ small instances and $3 \cdot 3 \cdot 3 \cdot 1 = 81$ large instances, respectively. We presuppose a typical container yard setup with 100 slots, 6 train tracks and equally sized crane areas. Each single instance is generated as follows: First, we apply $\bar{n}$ and $SPLIT$ to deduce the number of direct moves $Rd((1 - SPLIT) \cdot \bar{n})$ and the number of split/sorter moves $Rd(SPLIT \cdot \bar{n})$ (where $Rd(x)$ is the nearest integer to x). Then, random moves from train tr_j and slot sl_j to train tr'_j and slot sl'_j defined by $[(tr_j, sl_j), (tr'_j, sl'_j)]$ are generated. In order to create a direct move, start and ending position of the move are randomly chosen from the same crane area, which is chosen randomly as well. Split moves are constructed by picking start and ending position from different (randomly chosen) crane areas. As each split move requires an in-move ($[(tr_j, sl_j), (tr^*, sl_j)]$, with sorter track tr^*), an out-move ($[(tr^*, sl'_j), (tr'_j, sl'_j)]$), and a sorter move ($[(tr^*, sl_j), (tr^*, sl'_j)]$), we obtain a total of $n = |J| = \bar{n} + 2 \cdot Rd(SPLIT \cdot \bar{n})$ moves to be executed. If predecessors exist, that is $\exists i, j \in J$ with $(tr_i, sl_i) = (tr'_j, sl'_j)$, we add i to the set of j's predecessors Γ_j. In order to reduce complexity, we ensure, that cycles within sequences of moves are prevented. If cycles occur in real-life problems, an additional move can be introduced, which places one container of the cyclic sequence temporarily in the storage area.

Table 2 Parameters for instance generation

Symbol	Description	Values	
		Small	Large
$\bar{n}$	Number of initial moves	6, 7, 8, 9, 10, 11, 12, 13, 14, 15	50, 75, 100
m	Number of cranes	2, 3, 4	2, 3, 4
o	Number of shuttles	1, 2, 3	1, 2, 3
$SPLIT$	Percentage of split moves	5, 10, 15, 20, 25	10

Table 3 Parameters and common values for time computation

Symbol	Description	Conventional values
d^h	Distance between two slots (longitudinal)	14 m
d^v	Distance between two trains (lateral)	7 m
d^s	Spreader distance (height from ground level to steeple cab)	10 m
v^h	Crane speed (longitudinal)	3 m/s
v^v	Steeple cab speed (lateral)	2 m/s
v_l^s	Speed of loaded spreader	0.5 m/s
v_u^s	Speed of unloaded spreader	1 m/s
v^a	Speed of automatic guided sorter vehicle	2 m/s
pos^h	Time to position crane (longitudinal)	2 s
pos^v	Time to position steeple cab (lateral)	2 s
pos_l^s	Time to position loaded spreader	4 s
pos_u^s	Time to position unloaded spreader	6 s

In a second step, we utilize the parameters given in Table 3 to determine

- pick-up/drop-down time

$$\delta^{\text{in}} = \frac{d^s}{v_l^s} + pos_l^s + \frac{d^s}{v_u^s} \quad \text{and} \quad \delta^{\text{out}} = \frac{d^s}{v_u^s} + pos_u^s + \frac{d^s}{v_l^s}, \tag{32}$$

- job processing times p_j

$$p_j = \begin{cases} 0, & \text{if } j = 0 \\ \delta^{\text{out}} + \max\left\{ \frac{|sl_j - sl'_j| \cdot d^h}{v^h} + pos^h, \frac{|tr_j - tr'_j| \cdot d^v}{v^v} + pos^v \right\} + \delta^{\text{in}}, & \text{if } j \in J^C \\ \delta^{\text{out}} + \frac{|sl'_j - sl_j| \cdot d^h}{v^a} + \delta^{\text{in}}, & \text{if } j \in J^V \end{cases} \tag{33}$$

and

- setup times τ_{ij}

$$\tau_{ij} = \begin{cases} \max\left\{ \frac{|sl_j - sl'_i| \cdot d^h}{v^h} + pos^h, \frac{|tr_j - tr'_i| \cdot d^v}{v^v} + pos^v \right\}, & \text{if } \exists c \in C : i, j \in J_c \cup \{0\}, i \neq j \\ \frac{|sl_j - sl'_i| \cdot d^h}{v^a}, & \text{if } i, j \in J^V, i \neq j \\ \tau_{0j} = \tau_{j0} = 0, & \text{if } j \in J^V \end{cases} . \tag{34}$$

Note that the parameters listed in Table 3 have been received from a currently running container yard in Germany, so that the resulting crane movement times are close to practical applications. Our large instances are designed, such that they represent a 2-h working interval.

The described algorithms have been implemented in VB.NET and all tests have been performed on an Intel Core 2.5 GHz i5-2520M notebook with 4GB memory.

Table 4 Performance of solution procedures for small instances

$\bar{n}$	#opt	#unf	Avg. makespan gap (%)				Avg. cpu time (s)				
			NN	S1	S2	S3	NN	S1	S2	S3	XP (opt)
6	225	0	13.62	0.18	0.18	0.06	<0.01	0.29	<0.01	3.26	0.07
7	225	0	13.27	0.20	0.20	0.16	<0.01	0.35	<0.01	3.91	0.11
8	225	0	13.39	0.39	0.39	0.11	<0.01	0.43	<0.01	4.28	0.19
9	225	0	14.18	0.40	0.40	0.08	<0.01	0.50	<0.01	4.75	0.36
10	222	3	15.36	0.75	0.75	0.24	<0.01	0.60	<0.01	4.91	2.08
11	223	2	18.19	0.93	0.93	0.39	<0.01	0.72	<0.01	6.79	6.85
12	201	24	16.88	1.08	1.08	0.46	<0.01	0.83	<0.01	6.57	20.44
13	196	29	19.98	1.28	1.28	0.83	<0.01	0.95	<0.01	7.13	19.54
14	164	61	20.72	1.70	1.70	0.78	<0.01	1.08	<0.01	7.48	32.39
15	146	79	23.73	1.91	1.91	1.24	<0.01	1.22	<0.01	7.62	31.31
Total			16.52	0.81	0.81	0.39	<0.01	0.66	<0.01	5.51	9.73

First, we test the solution performance of our decomposition procedure for the small instances. Our benchmark is standard solver 'Xpress MP' (XP) solving the mixed-integer formulation given in Sect. 2.2 with a time limit of 300s, which should be sufficient for small instances of an operational decision problem. As Table 4 reveals, only for very small instances with 6 to 9 moves, XP is able to find any optimal solution (column #opt). Increasing the number of moves leads to more unfinished instances (#unf). The table also provides the average optimality gap (in %) and average cpu times (in seconds) for all those instances solved to optimality by XP for each stage of our decomposition procedure (S1, S2, S3) and for a nearest neighbor heuristic (NN). NN resembles our CH approach, but always chooses the next job according to the smallest time (instead randomly with different probabilities). It is, thus, the deterministic version of our CH approach and returns just a single solution. In our study, it serves as a further benchmark for our sophisticated decomposition procedure, because in real-life container terminals optimization seldom goes beyond simple decision rules (Boysen et al. 2010). In fact, most terminals we visited do not even apply any computerized scheduling procedure but leave the scheduling decisions to the crane operators. Note that we have applied the following parameter setting for our algorithm. We repeat CH $\psi = 50,000$ times (S1), apply the exact DP approach in stages S2 and S3, and set stage 3 parameters to $MOD = \{5, 10\}$, $REP = 5$, and $TOL = 10$.

The results show, that stage 1 is able to quickly derive competitive solutions. Within an average of less than 2s, an optimality gap of only 0.81 % is achieved. Obviously, stage 1 profits from the large number of solutions that can be evaluated in very short time. Stage 2, however, hardly comes in handy with cpu times less than 0.05s for every instance. As stage 1 already provides close to optimal solutions, DP is seldom able to improve job sequences. Stage 3, on the other hand, is able to improve solution values and reduces the optimality gap to an average of 0.39 %, but takes a few seconds of computational time. It can be concluded that for small instances our decomposition approach provides near optimal solutions significantly better than the simple NN

Table 5 Performance of solution procedures for large instances

$\bar{n}$	#fsb	Gap to NN (%)				cpu time (s)			
		S1	S2	S3	XP (feas)	NN	S1	S2	S3
50	24	9.53	9.53	11.47	−8.95	<0.01	0.28	0.25	10.97
75	5	10.20	10.74	12.63	−27.45	<0.01	0.69	1.20	23.84
100	1	5.93	6.07	7.76	−5.69	<0.01	1.31	1.04	56.37
Total	30	8.56	8.78	10.62	−11.93	<0.01	0.76	0.83	30.39

heuristic, whereas computational times are considerably smaller than those of an off-the-shelf solver.

These trends continue for the large instances (see Table 5). Here, we repeat CH $\psi = 1000$ (S1), apply beam search in stages 2 and 3, and set parameters to $BW = 500$ (S2), $MOD = \{5, 6\}$, $REP = 3$, $SBW = 25$, $LBW = 50$ and $TOL = 10$ (S3). Preliminary tests have shown that these parameter settings provide acceptable solutions given the typical time-performance trade-off. With a time limit of 600 s, XP was not able to find a feasible solution in every instance. The first column displays the number of instances (out of 27 for each number of moves) in which XP was able to find a feasible solution (#fsb). Furthermore, Table 5 summarizes the relative improvement of the objective value (in %) of XP and the three stages of our decomposition procedure compared to the results of the Nearest Neighbor heuristic and gives some indication of computational times. Again, our procedure outperforms the simple NN heuristic and stage 3 provides the most promising results. On average, 11.2 min of operational time can be saved by S3 during a 2-h working interval. Thus, even instances of real-world size are solvable by our decomposition approach in a reasonable amount of time (less than half a minute on average) and the results are considerably better than those derived by simple decision rules such as NN. Note that XP (with a time limit of 600 s) is outperformed by NN by an average of 11.93 %.

5 Performance analysis for different design and layout alternatives

Once a suited solution procedure for solving CSCSP is available it can be applied to investigate the effects of different layout and design alternatives on the yard performance. Specifically, we focus different choices to be made during the layout and design phase, such as the number and speed of cranes and sorter vehicles and the location of the sorter. For all these alternatives, we vary the values for these design parameters in a sensitivity analysis in order to identify their impact on the operational yard performance. The intention is to support yard managers identifying suited choices for each design parameter during the layout phase of a terminal. This way, they are able to trade off operational performance against the costs of a design alternative. A crucial choice in this context is a well-balanced crane and shuttle fleet. Adding extra shuttles increases yard performance only up to a certain level where all cranes are loaded to capacity. Adding more cranes increases the division of labor, but also

Table 6 Parameters for layout variations

Parameter	Values (defaults tagged with *)
Number of moves	10, 20, 30, 40, 50*, 60, 70, 80, 90, 100
Distance of moves (slots)	0–10, 10–20*, 20–30, 30–40
Number of cranes	1, 2, 3*, 4, 5, 6
Speed factor of cranes	0.25, 0.5, 1*, 2, 4
Number of sorter vehicles	1, 2, 3, 4, 5, 6, 7, 8, 9, 10*, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
Speed factor of sorter vehicles	0.25, 0.5, 1*, 2,4
Sorter position	Outside*, center

increases the number of split moves. Therefore, a concerted planning of the crane and shuttle fleet seems most important and a second intention of this section is to derive a suited planning approach to reach this goal.

As a basis for our test, we consider a typical German container yard with a length of 700m separated into 100 slots for standard TEU containers and six rail tracks. During our analysis we alter parameters that can be divided into three categories: input move properties (number and length of container moves), crane properties (number and speed of cranes), and sorter properties (number and speed of shuttles and sorter position). We examine the influence of each parameter separately, while all other parameters are set to a given default value (see Table 6).

For each set of parameters, instances are generated as described in Sect. 4 with two exceptions. First, our randomly generated moves have a given distance drawn from a given interval (see Table 6). This distance affects the number of split moves: Longer moves have a higher probability to be executed as split moves than short ones. As crane areas are equally sized and a default of three cranes is applied, a maximum distance of 40 slots for each move is sufficient for our study; longer moves will always result in split moves and can, thus, be neglected. Furthermore, we alter the position of the sorter. 'Center' represents a sorter in the middle of the yard as is depicted in Fig. 1a and 'outside' denotes a layout where the sorter is located at the outside of all six rail tracks. Note that the speed factor for the cranes affects the speed of the crane and spreader and is simply multiplied with the default speeds defined in Table 3, whereas the factor of the sorter vehicles only affects the shuttle car speed (see also Table 3). For each set of parameters, we have repeated instance generation 100 times and have solved each resulting instance with our decomposition approach with $\psi = 1000$ (S1), $BW = 100$ (S2), $MOD = \{5\}$, $REP = 1$, $SBW = 25$, $LBW = 50$ and $TOL = 10$ (S3).

For each solution derived with our decomposition approach, we also calculate the minimum shuttle fleet that would have been sufficient to timely execute all split moves without delay. As is defined in Table 6 the default value for the shuttle fleet is ten vehicles. However, in some instances not all these shuttles are actually required and a smaller shuttle fleet would have been sufficient without delaying any crane move. We calculate the minimum shuttle fleet size for executing all fixed sorter moves with the help of a maximum matching algorithm for bipartite graphs (see Bertossi et al. 1987).

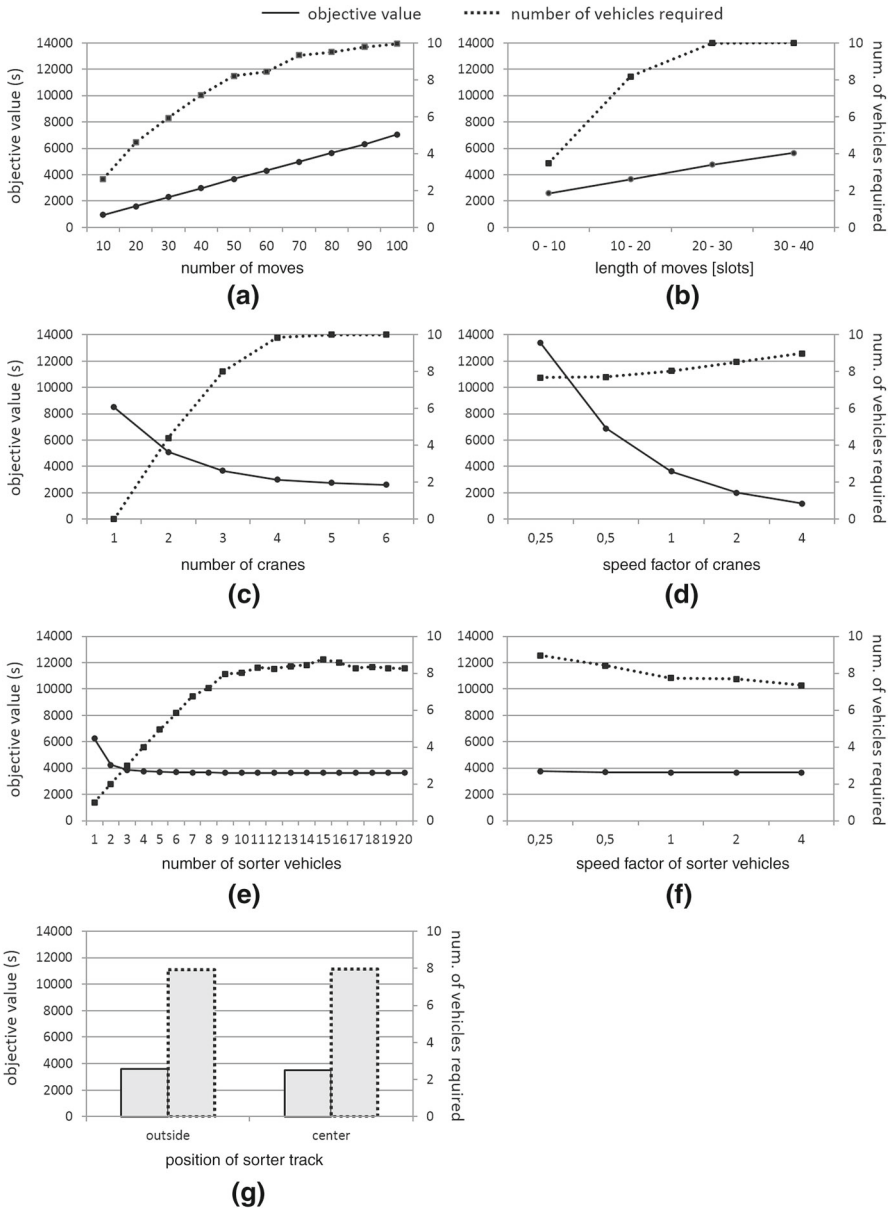

Fig. 6 Results when varying specific yard properties

This way, the minimum fleet size can be calculated in polynomial time. In addition to the makespan the required shuttle fleet size is another performance indicator. Figure 6 relates these indicators to different move, crane and shuttle characteristics:

(a) **Number of container moves:** The completion time increases linear in the number of container moves, with 100 moves being manageable in about 7052.8s = 1.96h.

This coherency allows to easily predict the makespan for different workloads. The number of shuttle cars to timely supply the gantry cranes increases with the number of moves. The more container moves there are the larger is the probability that some unfavorable capacity situation occurs, which can only be resolved by additional shuttles.

(b) **Distance of container moves:** Crane completion times increases proportional to the distance of container moves. Longer moves result in a higher amount of split moves and, therefore, double handling causes additional processing time. With a distance of 20–30 (30–40) slots about 33 % (48 %) of container moves are to be transferred between different crane areas, so that additional shuttles are required with increasing move distance.

(c) **Number of cranes:** Additional cranes reduce the average completion time of all jobs; however, their impact quickly diminishes. A second crane saves 40 % of completion time, a third crane 28 %, and an extra sixth crane only saves about 5 % of the makespan. Moreover, it is to be considered that additional cranes also require additional shuttles to timely supply them. Thus, decisions on the appropriate number of cranes and shuttles cannot be made independent of each other, but need to be carefully concerted.

(d) **Speed of cranes:** Faster cranes accelerate train processing and, thus, lead to a lower makespan. Again, we can see diminishing returns and accelerating a slow crane shows more impact than accelerating an already fast one. Furthermore, yard managers have to consider that faster cranes increase the shuttle fleet.

(e) **Number of sorter vehicles:** Considerable improvements in performance are only detected for the first (32 %), second (9 %), and third (3 %) additional sorter vehicles. More vehicles do not lead to significant changes in completion time. Once all containers are quickly transported towards the cranes additional, improvement can only be obtained by also increasing the crane number. Going beyond this level is useless. Additional vehicles just remain idle and do not affect the overall performance.

(f) **Speed of sorter vehicles:** The speed of shuttle cars has almost no impact on the performance of the yard. In optimized plans, split moves are closely synchronized anyway, so that faster shuttles cannot relieve the cranes. The only considerable impact is on the shuttle fleet. Faster shuttles can execute more jobs in the same time, so that (on average) the fleet size can be reduced.

(g) **Sorter position:** The sorter can either be located in the center as is depicted in Fig. 1a or under all six rail tracks. Both sorter positions, however, show only negligible influence on the completion time and the shuttle fleet. This is due to two competing effects: A central sorter reduces the crane distances for the split moves to and from the sorter. Direct moves, however, require increasing crane travel, whenever the crane has to cross the central sorter. Both effects seem to outweigh each other, so that there is no clear recommendation for either sorter position.

A major conclusion from these computations is that a yard's crane configuration cannot be determined independently of its shuttle fleet. In the following, we, thus, sketch a procedure of how to determine an appropriate and harmonized crane and

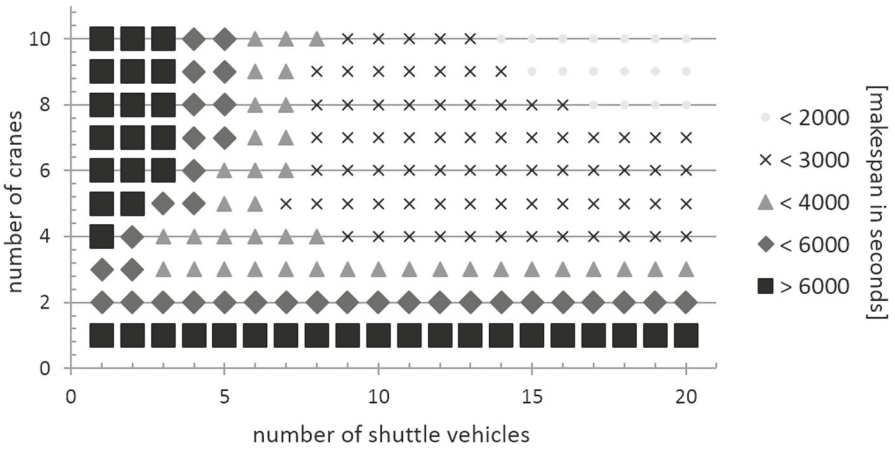

Fig. 7 Makespan chart to identify efficient yard layouts

shuttle fleet for a specific yard setting. First hand, we presuppose that the velocity of cranes and shuttles is beyond a yard manager's area of influence. Typically, there is some current state of technology and crane and shuttle producer offer their vehicles according to a given technological specification. If, however, different cranes and shuttles with varying velocity are offered, then our following makespan chart needs to integrate velocity as well (i.e., in a three dimensional representation) or needs to be recalculated for different states of technology. For a matter of conciseness, we assume a given state of technology, i.e., crane and shuttle speed. Then, a representative workload is to be identified, which contains the right number and proportion of split and direct moves. For such a representative workload, we can calculate the crane and shuttle schedules by solving our decomposition approach. For deriving the makespan chart of Fig. 7, we have solved 25 instances per parameter setting according to the default values defined in Table 6. For each of these 25 instances we calculate the average makespan (in seconds) and the average shuttle fleet required to timely execute all sorter moves (see above). Figure 7 summarizes these results by relating the crane number and shuttle fleet with the resulting level of makespan.

With the help of such a makespan chart a terminal manager is able to identify a balanced terminal configuration. Clearly, the more cranes and shuttles are added the faster is the train processing. Therefore, a terminal manager has to identify an acceptable makespan for his/her representative workload and can, finally, identify an efficient crane number and shuttle fleet size. Note that the cost relationship between cranes and shuttles need to be considered here, because (at least partially) cranes can be substituted by shuttles and vice versa.

The makespan chart of Fig. 7 reveals another interesting coherency. Presumably, one would rather expect some properly shaped isoquant, but here there is a distinct dent in crane direction, which is due to the following reasons:

- Adding an extra shuttle increases performance whenever the waiting time of cranes is reduced. Once, all cranes are permanently occupied without idle time adding extra shuttles has no further performance impact (neither positive nor negative).

- This simple coherency does not hold for the cranes. Adding a crane to a balanced capacity situation (i.e., enough shuttles to steadily occupy all cranes) leads to an additional crane area, which increases the number of split moves to be processed. With these additional split moves, the existing shuttle fleet might not be sufficient anymore to timely supply all cranes. The result is that adding a crane may actually deteriorate yard performance. Only if additional shuttles are added all cranes can operate at full capacity again and the makespan is reduced.

Thus, finding a balanced yard configuration where there are enough shuttles to steadily occupy all cranes is not a trivial matter. Our makespan chart depicted in Fig. 7 is a enabler to resolving this planning task in an informed manner.

6 Conclusions

This paper treats the integrated scheduling of gantry cranes and shuttle vehicles in modern rail–rail transshipment yards. In these yards, shuttle vehicles are applied to relieve the cranes from longitudinal movement along the spread of the yard, so that a crane sets down a container on a shuttle, which moves the box into the target area where, finally, another crane transfers the container from the shuttle onto the target railcar. These split moves via the sorting system require a careful synchronization of cranes and shuttle vehicles when determining operational schedules. We formalize the resulting interdependent gantry crane and shuttle car scheduling problem (CSCSP), prove its computational complexity, and provide a heuristic decomposition procedure for its solution. The application of this solution approach in our computational study reveals the following take home messages:

- The position of the sorter either in the center of the yard or at the outside has no considerable impact on yard performance. The reduced crane travel for the split moves to and from the sorter in case of a central sorter are directly outweighed by the additional effort for direct moves going across the sorter and vice versa.
- Finding a balanced yard configuration where there are enough shuttles to steadily occupy all cranes is among the most elementary design issues. Adding extra shuttles only pays up to a level where cranes do not operate at full capacity and adding cranes may actually deteriorate yard performance due to the additional crane area and the extra split moves. The makespan chart suggested in Sect. 5 is a simple matter to identify well-balanced yard configurations.

On the one hand, future research should challenge our solution procedures and develop new exact and heuristic solution approaches. Especially exact solution procedures are a challenging future research task. A major prerequisite of our problem setting is the existence of fixed crane areas each operated by its dedicated crane. Evaluating the impact of this choice compared to free moving cranes (nonetheless bound to non-crossing constraints) would be valuable decision support for yard managers having to opt for or against fixed crane areas.

Appendix A: Complexity of CSCSP

In this section we prove NP-hardness in the strong sense of CSCSP. At first sight, this complexity status directly follows from its relation to the asymmetric traveling salesman problem (ATSP). If we have neither split moves nor precedence constraints, our problem decomposes into m ATSPs. This relationship, however, is not sufficient for proving NP-hardness. All start and target positions of container moves lie along parallel lines, i.e., on the tracks or the sorter. This results in specially structured distance matrices and there are plenty special cases of the traveling salesman problem with similar prerequisites that turn out being solvable in polynomial time (see Burkard et al. 1998 for a survey).

Our reduction is from the 3-Partition problem instead, which is well-known to be strongly NP-hard (Garey and Johnson 1979).

3-Partition: Given $3q$ integers $a_1, \ldots, a_{3q}$ with $\frac{B}{4} < a_j < \frac{B}{2}$. Does there exist a partition of set $\{1, 2, \ldots, 3q\}$ into q subsets $A_1, \ldots, A_q$ such that $\sum_{j \in A_i} a_j = B$ for each $i = 1, \ldots, q$?

The transformation scheme for an instance of 3-Partition to an instance of CSCSP is as follows: We have $m = 2$ cranes and a single shuttle vehicle $o = 1$. The job set contains $q + 1$ split moves going from crane 2 to crane 1 and $3q + 1$ direct moves all located in the area of crane 1. Among these direct moves are $3q$ moves that build the *counterpart jobs* corresponding to the integer values of 3-Partition. Additionally, we have a single *start move*. The shuttle move of each of the $q + 1$ split moves takes $\frac{B}{2}$ time units, whereas of the belonging in- and out-moves have negligible processing time. The processing times of the counterpart jobs equal $p_j = \frac{a_j}{2}$, i.e., half of the integer values they refer to. Setup time τ_{ji} also equals $\frac{a_j}{2}$. Note that all counterpart jobs are assumed to start along the same vertical line, the *start line*, so that executing a job j and moving back to the start line takes the crane a_j time units. The single start job has a processing time of $\frac{B}{2}$ and pickup (δ^{out}) as well as set-down times (δ^{in}) equal zero. The question we ask is whether a makespan of $Z \leq (q + \frac{1}{2}) \cdot B$ can be realized.

The given makespan induces that the shuttle vehicle has to be loaded instantaneously in crane area 2 and then continuously moves between both crane areas as it is depicted in Fig. 8a. Otherwise the given bound Z on the makespan will inevitably be exceeded. The total workload of crane 1 equals the makespan, so that idle time must not occur. The only possibility to fill the time span up to the first arrival of the shuttle at crane area 1 without idle time is the execution of the start move. Due to the restrictions of the integer values of 3-Partition no subset of direct moves fits into this gap. To not delay the shuttle vehicle crane 1 has to process it instantaneously upon arrival. This leaves exactly q gaps of B time units each for scheduling the counterpart jobs between any arrival of the shuttle and the one-to-one mapping between both problems is directly available.

The only remaining aspect to detail is how the considered processing times of the container moves can occur given a container yard setting (see Fig. 8b). We assume that each crane moves one distance unit per time unit in longitudinal direction and has infinite velocity in lateral direction. All direct (split) moves have their start (target) positions along the start line, which is also the initial start position of crane 1. To

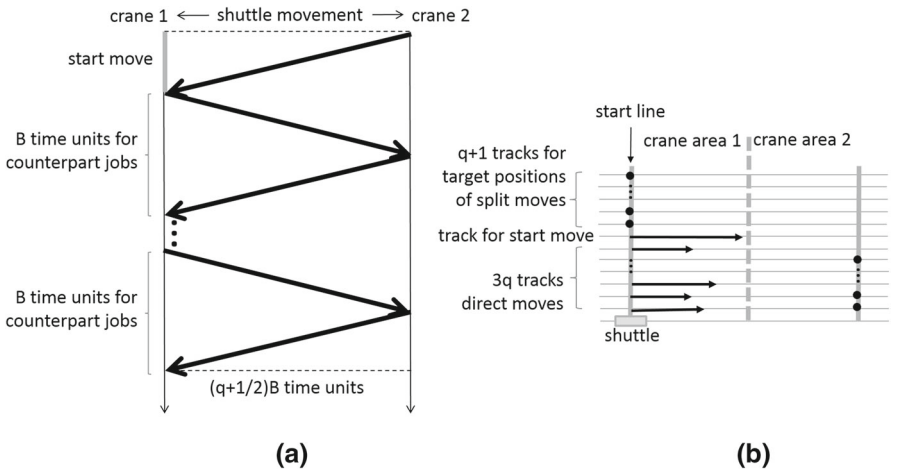

Fig. 8 Schedule structure of a transformed CSCSP instance

give all container moves enough space along the start line we, thus, have to introduce $4q + 2$ tracks. Along the start line, the crane moves with infinite velocity, so that the out-moves require no processing time. Time is only consumed by the direct moves, which require the way from the start line towards their respective target position (i.e., their processing time) and the setup time back to the start line in order to process another move.

Appendix B: Complexity of MFP

In this section, we prove that the mode feasibility problem applied in stage 3 of our decomposition approach (see Sect. 3.3) is strongly NP-complete even if $|V| = 1$, i.e., only a single shuttle is available.

Consider an infinite velocity of cranes, so that $\Psi_{\mu j}^{Cr} = \emptyset$ for all $\mu \in M_j$ and $j \in J^V$ and only a single shuttle, i.e., $|V| = 1$, then MFP equals the following discrete interval scheduling problem (DISP), which was shown to be strongly NP-complete by Nakajima and Hakimi (1982): consider a set J of jobs to be processed on a single machine. Each job $j \in J$ is assigned a set of alternative processing intervals (or modes). If a specific processing interval of a job is selected, this means that during the complete time span from the fixed start to the end of the interval the machine is occupied by this job. The DISP decides whether all jobs can be processed, i.e., exactly one mode per job is selected, on a single machine without overlap. MFP is a generalization of this problem, which completes the proof.

References

Alicke K (2002) Modeling and optimization of the intermodal terminal Mega Hub. OR Spectr 24:1–17

Alicke K, Arnold D (1998) Modellierung und Optimierung von mehrstufigen Umschlagsystemen. Fördern und Heben 8:769–772
Ambrosino D, Siri S (2015) Comparison of solution approaches for the train load planning problem in seaport terminals. Transp Res Part E 79:65–82
Bektas T (2006) The multiple traveling salesman problem: an overview of formulations and solution procedures. Omega 34:209–219
Bertossi AA, Carraresi P, Gallo G (1987) On some matching problems arising in vehicle scheduling models. Networks 17:271–281
Bierwirth C, Meisel F (2010) A survey of berth allocation and quay crane scheduling problems in container terminals. Eur J Oper Res 202:615–627
Bierwirth C, Meisel F (2015) A follow-up survey of berth allocation and quay crane scheduling problems in container terminals. Eur J Oper Res 244:675–689
Bontekoning Y, Priemus H (2004) Breakthrough innovations in intermodal freight transport. Transp Plan Technol 27:335–345
Bostel N, Dejax P (1998) Models and algorithms for container allocation problems on trains in a rapid transshipment shunting yard. Transp Sci 32:370–379
Boysen N, Fliedner M (2010) Determining crane areas in intermodal transshipment yards: the yard partition problem. Eur J Oper Res 204:336–342
Boysen N, Fliedner M, Kellner M (2010) Determining fixed crane areas in rail-rail transshipment yards. Transp Res Part E Logist 46:1005–1016
Boysen N, Jaehn F, Pesch E (2011) Scheduling freight trains in rail-rail transshipment yards. Transp Sci 45:199–211
Boysen N, Fliedner M, Jaehn F, Pesch E (2012) Shunting yard operations: theoretical aspects and applications. Eur J Oper Res 220:1–14
Boysen N, Jaehn F, Pesch E (2012) New bounds and algorithms for the transshipment yard scheduling problem. J Sched 15:499–511
Boysen N, Fliedner M, Jaehn F, Pesch E (2013) A survey on container processing in railway yards. Transp Sci 47:312–329
Boysen N, Briskorn D, Meisel F (2014) A generalized classification scheme for crane scheduling with interference. Working Paper Friedrich-Schiller-University Jena
Bredström D, Rönnqvist M (2008) Combined vehicle routing and scheduling with temporal precedence and synchronization constraints. Eur J Oper Res 191:19–31
Bruns F, Knust S (2012) Optimized load planning of trains in intermodal transportation. OR Spectr 34:511–533
Bruns F, Goerigk M, Knust S, Schöbel A (2014) Robust load planning of trains in intermodal transportation. OR Spectr 36:631–668
Burkard RE, Deineko VG, van Dal R, van der Veen JA, Woeginger GJ (1998) Well-solvable special cases of the traveling salesman problem: a survey. SIAM Rev 40:496–546
Castillo-Salazar JA, Landa-Silva D, Qu R (2016) Workforce scheduling and routing problems: literature survey and computational study. Ann Oper Res 239:39–67
Delta Air Lines (2013) Stats and Facts. Delta Air Lines homepage. http://news.delta.com/index.php?s=18&cat=47. Retrieved 10 Oct 2013
Drexl M (2012) Synchronization in vehicle routing—a survey of vrps with multiple synchronization constraints. Transp Sci 46:297–316
EU (2007) Mitteilung der Kommission an den Rat und das europSische Parlament: Aufbau eines vorrangig fnr den Gnterverkehr bestimmten Schienennetzes. Brnssel
Froyland G, Koch T, Megow N, Duane E, Wren H (2008) Optimizing the landside operation of a container terminal. OR Spectr 30:53–75
Gambardella LM, Dorigo M (2000) An ant colony system hybridized with a new local search for the sequential ordering problem. INFORMS J Comput 12:237–255
Garey MR, Johnson DS (1979) Computers and Intractability: A Guide to the Theory of NP-completeness. Freeman, New York
Held M, Karp RM (1962) A dynamic programming approach to sequencing problems. SIAM J 10:196–210
Kellner M, Boysen N, Fliedner M (2012) How to park freight trains on rail-rail transshipment yards: the train location problem. Oper Res Spectr 34:535–561
Kellner M, Boysen N (2015) RMG vs. DRMG: an evaluation of different crane configurations in intermodal transshipment yards. EURO J Transp Logist 4:355–377

Martinez MF, Gutierrez IG, Oliveira AO, Arreche Bedia LM (2004) Gantry crane operations to transfer containers between trains: a simulation study of a Spanish terminal. Transp Plan Technol 27:261–284

Montemanni R, Smith DH, Rizzoli AE, Gambardella LM (2009) Sequential ordering problems for crane scheduling in port terminals. Int J Simul Process Model 5:348–361

Nakajima K, Hakimi SL (1982) Complexity results for scheduling tasks with discrete starting times. J Algorithms 3:344–361

Rotter H (2004) New operating concepts for intermodal transport: the mega hub in Hanover/Lehrte in Germany. Transp Plan Technol 27:347–365

Souffriau W, Vansteenwegen P, van den Berghe G, van Oudheusden D (2009) Variable neighbourhood descent for planning crane operations in a train terminal. In: Sörensen K, Sevaux M, Habenich W, Geiger MJ (eds) Metaheuristics in the service industry, Lecture Notes in Economics and Mathematical Systems, vol 624. Springer, Berlin, pp 83–98

Toth P, Vigo D (2002) The Vehicle Routing Problem. SIAM Monographs on Discrete Mathematics and Applications, Philadelphia

Trip JJ, Bontekoning YM (2002) Integration of small freight flows in the intermodal transport system. J Transp Geogr 10:221–229

Vahrenkamp R (2010) Logistik und Gntertransport in Europa 1950 bis 2000. Working Paper in the History of Mobility No. 13/2009 University of Kassel **(in German)**